

程序设计实习 大作业报告

小组成员：孙广侑 2500094811；徐至晟 2400015881

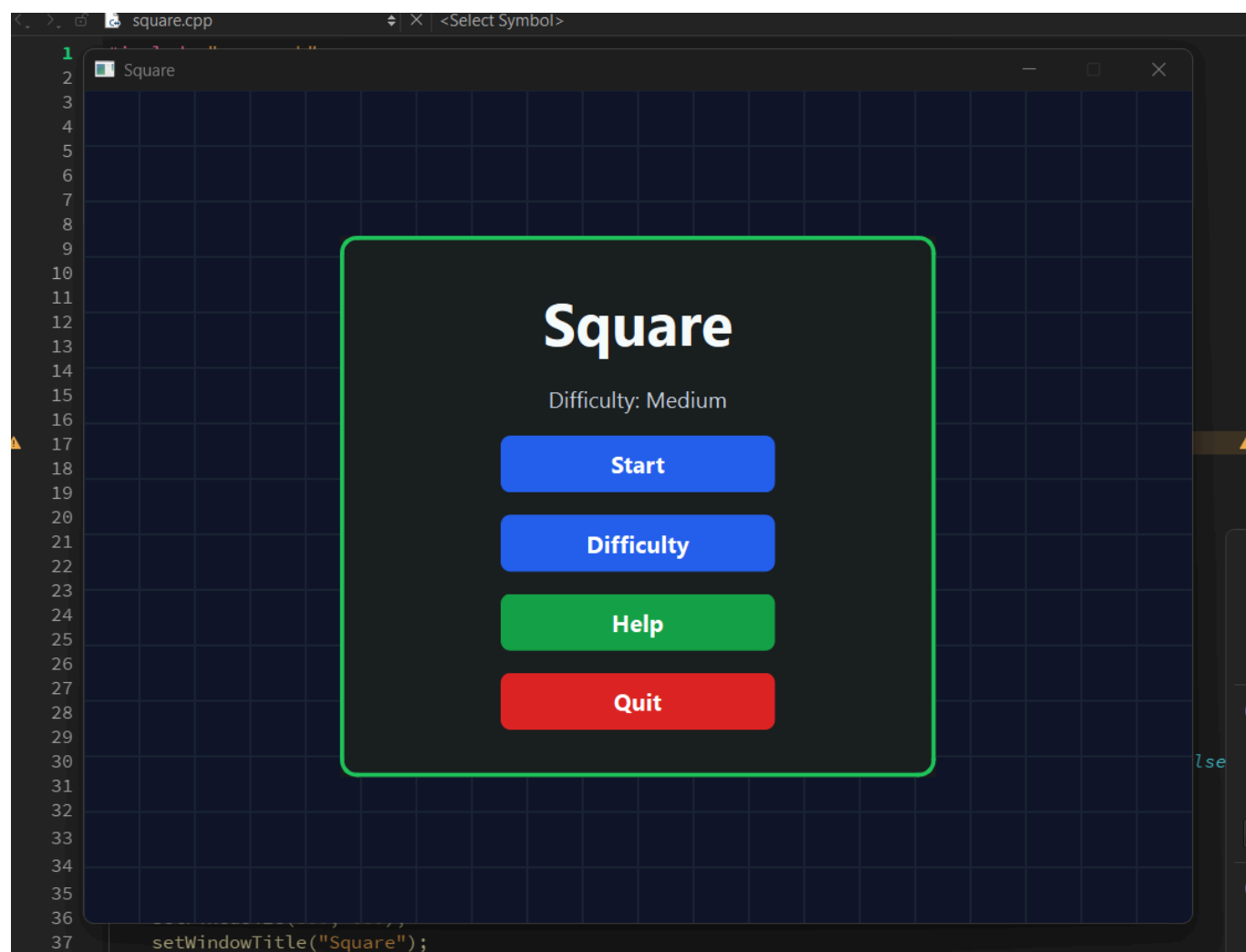
一、项目概述

项目名称

Square - 基于 Qt 的方块吞噬小游戏，游戏类方向。

项目简介

Square 是一款使用 C++ 和 Qt 开发的桌面小游戏。游戏采用简洁直观的方块画面，玩家通过键盘控制绿色方块移动，吞噬更小的敌人方块并不断成长，同时躲避更大的危险方块。项目在基础吞噬玩法上加入了多类型敌人、限时道具、难度系统、暂停菜单、胜负弹窗和实时状态栏等内容，使游戏具有较完整的可玩性和交互体验。



核心亮点

1. 多类型敌人系统

游戏设计了普通、追踪、闪烁、分裂、恐惧、神速等多种敌人类型，不同敌人具有不同运动方式和特殊效果，增加了游戏变化。

2. 道具与策略机制

玩家吞噬敌人后有概率生成道具球，可以获得屏障、减速场、磁铁、缩小炸弹等效果。道具既能帮助玩家突破困境，也带来一定风险和策略选择。

3. 安全生成与难度平衡

程序在敌人生成时检测重叠和玩家安全距离，并保证开局存在一定数量的小敌人，减少“开局必输”的情况。四种难度通过参数统一控制敌人数量、速度、大小和得分倍率。

4. 实时反馈与视觉提示

敌人边框颜色实时提示是否可吞噬：绿色表示安全，红色表示危险。不同敌人还使用不同颜色和图形符号进行区分，状态栏显示当前难度、玩家大小、剩余敌人数、得分和道具状态。

5. 完整的游戏流程

项目包含主菜单、难度选择、规则说明、暂停、失败、胜利等界面，形成了从开始游戏到结束反馈的完整流程。

程序功能介绍

本项目使用 C++ 和 Qt 实现了一个方块吞噬类小游戏。玩家通过键盘方向键控制绿色方块在窗口中移动，吞噬比自己更小的方块来增大体型；如果碰到比自己更大的方块，则游戏失败；当场上所有敌人方块都被消灭后，游戏胜利。

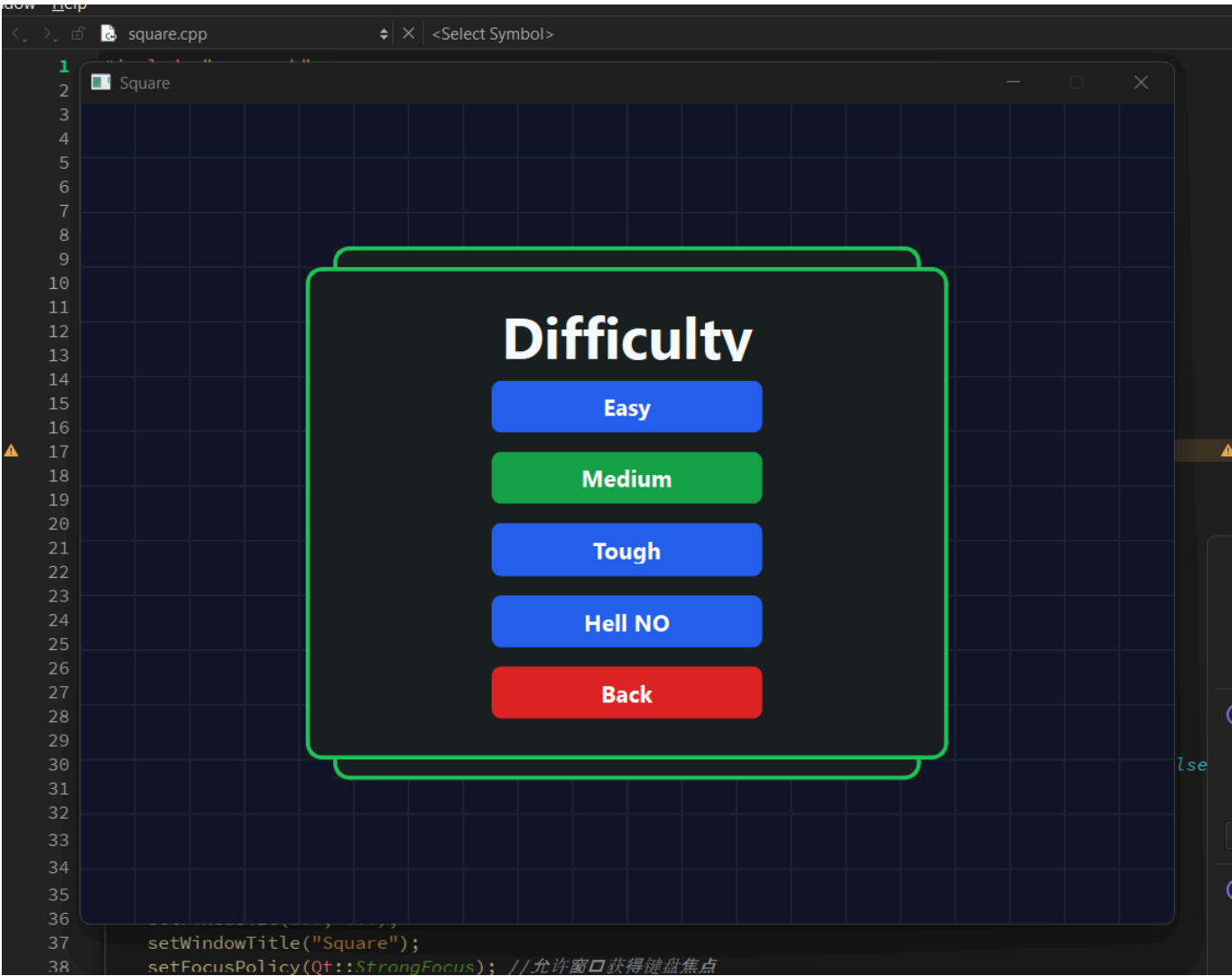
游戏通过敌人方块的边框颜色提示玩家当前是否可以吞噬：绿色边框表示可以吞噬，红色边框表示危险。为了提高游戏难度与趣味性，项目设计了多种敌人类型，包括沿直线运动的普通敌人、主动追踪玩家的敌人、大小周期性变化的敌人、被吞噬后会分裂的敌人等。

玩家吞噬敌人后，有一定概率生成带问号的道具球。玩家吃到道具球后，可以随机获得一种限时道具效果：

道具	效果
屏障	免疫一次来自更大方块的碰撞伤害
减速场	暂时减慢大部分敌人的移动速度
磁铁	自动吸收附近可吞噬的方块
缩小炸弹	暂时缩小较大的敌人，持续 3 秒后敌人恢复并变为原大小的 160%

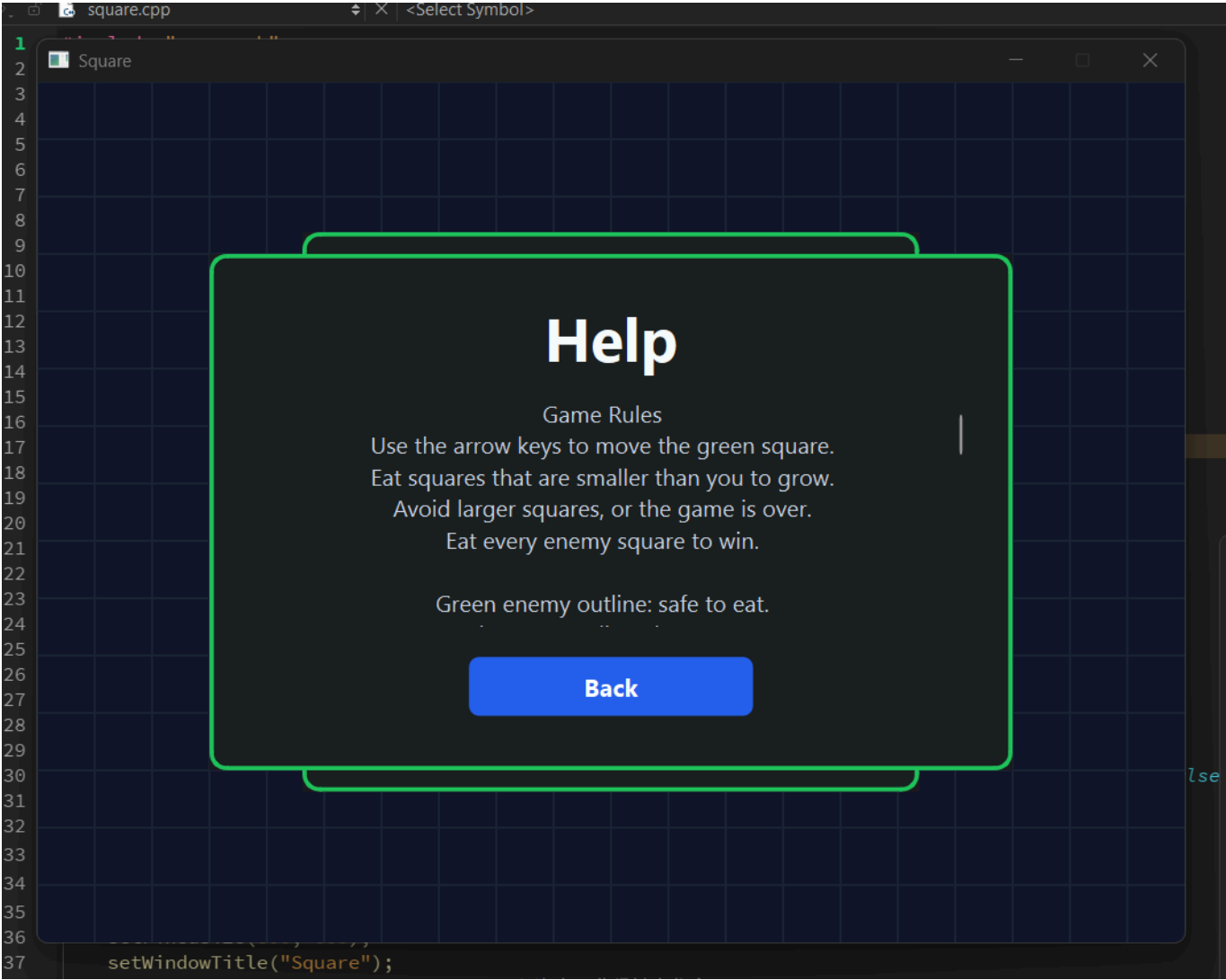
除基础移动外，玩家还可以按空格键进行短时间加速，加速存在冷却时间；按 Esc 键可以暂停游戏。

游戏提供四种难度：简单、中等、困难、地狱。难度越高，敌人数量越多，敌人的移动速度和体型也会相应提升。游戏界面包括主菜单、难度选择页面、规则说明页面和游戏主界面。游戏主界面上方会显示当前难度、玩家方块大小、剩余敌人数、得分以及道具激活状态等信息。游戏胜利、失败和暂停时，程序会弹出相应对话框提示玩家。



游戏操作说明

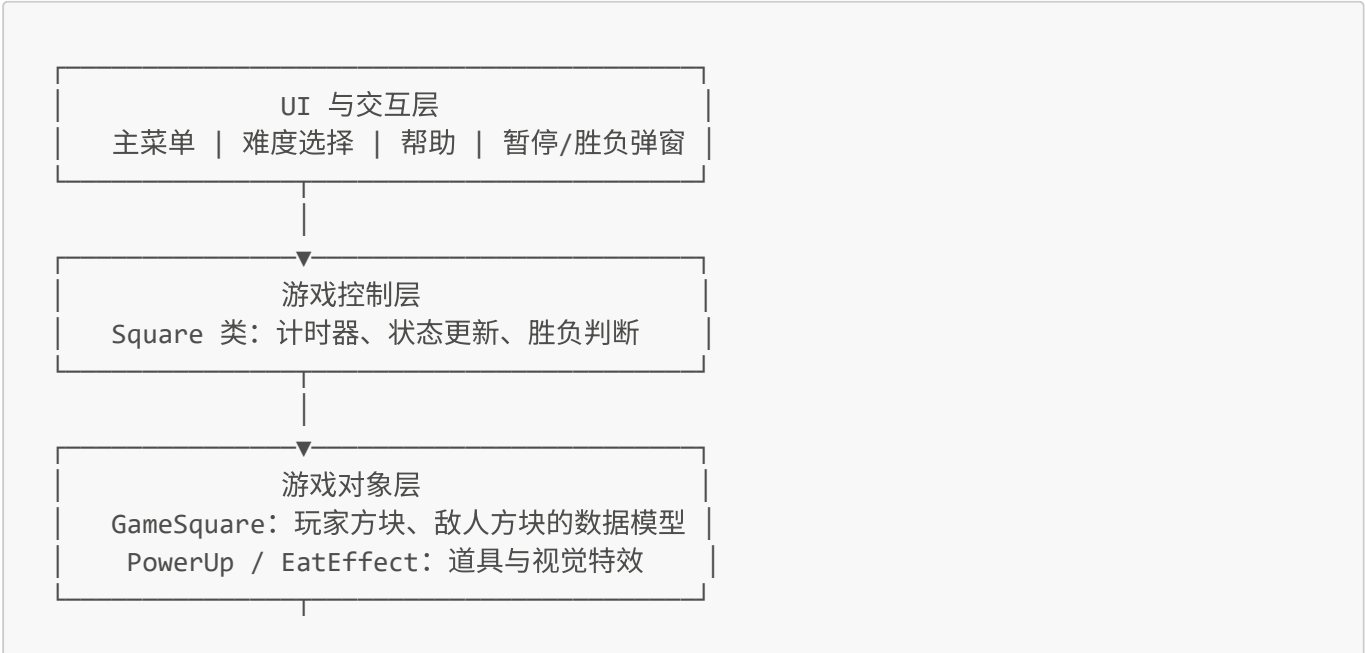
操作	功能
方向键 ↑ ↓ ← →	控制玩家方块向对应方向移动
空格键	短时间加速移动，加速后进入冷却
Esc	暂停游戏，弹出暂停菜单
鼠标点击菜单按钮	开始游戏、选择难度、查看规则或退出游戏

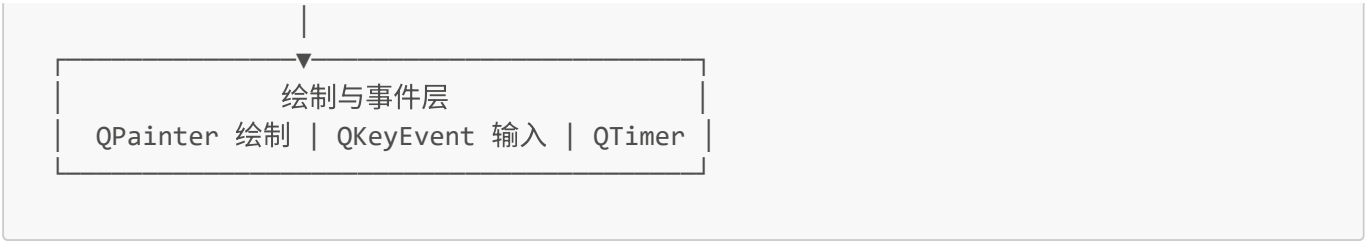


二、项目架构设计

2.1 总体架构

项目整体采用以 `Square` 主窗口类为核心的结构，将方块数据模型、游戏状态更新、绘制逻辑、输入事件和弹窗界面组织在一起。整体架构如下：





这种结构的特点是逻辑集中、调用关系清晰：`QTimer` 定时触发 `updateGame()`，游戏状态变化后调用 `update()` 刷新界面，`paintEvent()` 根据当前状态重新绘制画面。

2.2 项目模块与类设计

2.2.1 main.cpp

`main.cpp` 是 Qt 项目的主程序入口，负责创建应用程序对象并启动主窗口。

2.2.2 GameSquare.h 与 GameSquare.cpp

`GameSquare` 类用于表示玩家方块和敌人方块，是游戏中方块对象的基础模型。

主要成员变量包括：

变量类型	含义
坐标	方块在窗口中的位置
移动速度	方块在 X、Y 方向上的速度
边长	方块大小
颜色	方块显示颜色

主要成员函数包括：

函数	功能
构造函数	初始化方块位置、大小、速度和颜色
Get / Set 函数	读取或修改方块成员变量
<code>move()</code>	根据速度更新方块坐标
<code>toRect()</code>	将方块转换为 <code>QRect</code> ，便于使用 <code>intersects()</code> 判断碰撞

2.2.3 square.h 与 square.cpp

`Square` 类继承自 `QWidget`，是整个游戏的核心类，负责游戏状态管理、界面绘制、键盘输入、敌人逻辑、道具系统和胜负判断。

2.2.3.1 构造与析构

```
explicit Square(QWidget *parent = nullptr);
~Square();
```

构造函数负责初始化窗口、游戏计时器、菜单和初始游戏状态；析构函数负责释放相关资源。

2.2.3.2 事件处理函数

函数	功能
paintEvent(QPaintEvent *event)	绘制当前游戏画面，包括玩家、敌人、道具、特效和状态栏
keyPressEvent(QKeyEvent *event)	处理键盘按下事件，包括移动、暂停和加速
keyReleaseEvent(QKeyEvent *event)	处理键盘释放事件，更新当前按键状态

2.2.3.3 游戏状态初始化与更新

函数	功能
resetGameState()	重置游戏状态，将玩家放回中心，清空敌人、道具和特效
startGame()	重置游戏并初始化敌人，重新启动游戏计时器
updateGame()	游戏核心循环，负责更新玩家、敌人、道具移动和碰撞检测
restartGame()	重新开始游戏

2.2.3.4 对话框与界面

函数	功能
showMainMenu()	显示主菜单
showDifficultyDialog()	显示难度选择窗口
showHelpDialog()	显示规则说明
showPauseDialog()	显示暂停窗口
showGGDialog()	显示失败窗口
showVictoryDialog()	显示胜利窗口
centerDialog(QDialog& dialog) const	将弹窗显示在主窗口中央
dialogStyleSheet() const	返回统一的弹窗样式表

2.2.3.5 枚举与结构设计

项目使用枚举类型表示难度、敌人类型和道具类型，使代码逻辑更加清晰。

```
enum class Difficulty { Easy, Medium, Tough, HellNo };
enum class EnemyType { Normal, Hunter, Blinker, Splitter, Horror, Speedster };
enum class PowerUpType { Shield, SlowField, Magnet, ShrinkBomb };
```

其中：

类型	说明
Difficulty	游戏难度：简单、中等、困难、地狱
EnemyType	敌人类型：普通、追踪、闪烁、分裂、恐惧、神速
PowerUpType	道具类型：屏障、减速场、磁铁、缩小炸弹

DifficultySettings 结构体用于保存不同难度下的参数，例如敌人数量、基础大小、速度、玩家速度、分数倍率和最大道具数量等。

EatEffect 结构体用于记录吞噬敌人后的视觉特效，包括特效中心、颜色、持续时间和初始半径。

PowerUp 结构体用于表示道具球，包括中心坐标、道具类型、半径以及 X、Y 方向速度。

2.2.3.6 敌人生成与行为逻辑

敌人相关函数主要负责初始化、随机生成、碰撞检测、特殊行为和删除操作。

函数	功能
initEnemies()	初始化敌人
setRandomDirection(GameSquare& enemy)	为敌人设置随机移动方向
checkCollision(const GameSquare& enemy, int newX, int newY)	检查敌人与边界或其他对象的碰撞
checkEnemyCollisions(int index, int newX, int newY)	检查指定敌人与其他敌人的碰撞
isNearPlayer(const GameSquare& enemy, int margin) const	判断敌人是否接近玩家
steerAwayFromPlayer(GameSquare& enemy)	控制敌人远离玩家
steerHunterTowardPlayer(int index, int step)	控制追踪敌人靠近玩家
randomEnemyType(int index) const	根据难度和敌人序号随机生成敌人类型
enemyColorForType(EnemyType type) const	根据敌人类型返回显示颜色
appendEnemy(...)	添加敌人到列表
updateBlinkerEnemies()	更新闪烁敌人的大小变化
spawnSplitEnemies(...)	生成分裂敌人
mergedSizeAfterEating(...)	计算吞噬敌人后的玩家大小

函数	功能
<code>removeEnemyAt(int index)</code>	删除指定敌人

随着难度提升，特殊敌人的出现概率会提高，使游戏节奏更快、挑战性更强。

2.2.3.7 道具系统

道具系统负责道具球生成、移动、碰撞检测和效果激活。

函数	功能
<code>spawnPowerUp(const QPointF& center)</code>	在指定位置生成道具球
<code>setRandomPowerUpDirection(PowerUp& powerUp)</code>	设置道具球随机移动方向
<code>movePowerUps(int speed)</code>	更新道具球位置
<code>checkPowerUpCollisions()</code>	检测玩家是否吃到道具
<code>activatePowerUp(PowerUpType type)</code>	激活指定道具效果
<code>showPowerUpDialog(PowerUpType type)</code>	弹窗提示获得的道具
<code>powerUpName(PowerUpType type) const</code>	返回道具名称
<code>powerUpDescription(PowerUpType type) const</code>	返回道具说明
<code>applyMagnetEffect()</code>	应用磁铁效果
<code>applyShrinkBomb()</code>	应用缩小炸弹效果
<code>restoreShrinkBombEnemies()</code>	恢复被缩小炸弹影响的敌人
<code>updateVisualEffects()</code>	更新视觉特效

2.2.3.8 难度设置

函数	功能
<code>setDifficulty(Difficulty newDifficulty)</code>	设置当前难度
<code>currentSettings() const</code>	获取当前难度对应参数
<code>difficultyName() const</code>	返回当前难度名称

不同难度通过参数控制敌人数量、敌人体型、移动速度、玩家速度、分数倍率和道具数量，从而实现不同的游戏体验。

2.2.3.9 主要成员变量

成员变量	功能
<code>QTimer *gameTimer</code>	控制游戏循环刷新

成员变量	功能
<code>QSet<int> pressedKeys</code>	记录当前按下的键
<code>Difficulty difficulty</code>	当前游戏难度
<code>bool gameStarted</code>	游戏是否已经开始
<code>GameSquare player</code>	玩家方块
<code>QList<GameSquare> enemies</code>	敌人方块列表
<code>QList<EnemyType> enemyTypes</code>	敌人类型列表
<code>QList<int> enemyBaseSizes</code>	敌人基础大小列表
<code>QList<int> enemyPhaseOffsets</code>	敌人变化相位偏移
<code>QList<int> shrinkRestoreSizes</code>	缩小炸弹恢复大小列表
<code>QList<EatEffect> eatEffects</code>	吞噬视觉特效列表
<code>QList<PowerUp> powerUps</code>	道具列表
<code>int score</code>	当前得分
<code>int powerUpsDropped</code>	已掉落道具数量
<code>int playerPulseFrames</code>	玩家边框动画帧数
<code>int dashFrames</code>	加速持续帧数
<code>int dashCooldownFrames</code>	加速冷却帧数
<code>int shieldFrames</code>	屏障剩余帧数
<code>int slowFieldFrames</code>	减速场剩余帧数
<code>int magnetFrames</code>	磁铁剩余帧数
<code>int shrinkBombFrames</code>	缩小炸弹剩余帧数
<code>int gameFrame</code>	当前游戏帧编号

三、技术特点与实现

本项目的技术重点主要集中在实时游戏循环、碰撞检测、敌人行为设计、道具计时和 Qt 界面绘制等方面。

3.1 游戏循环与实时刷新

游戏使用 `QTimer` 驱动主循环，计时器每 16 毫秒触发一次 `updateGame()`，接近 60 FPS。每一帧中，程序依次完成玩家移动、敌人移动、道具移动、碰撞检测、胜负判断和画面刷新。这样的结构使游戏逻辑集中在统一的更新函数中，便于维护和调试。

3.2 敌人随机生成与安全距离控制

敌人生成时，程序会随机决定敌人的大小和位置，并使用 `QRect::intersects()` 判断新敌人是否与玩家安全区域或已有敌人发生重叠。生成位置不合适时会重新随机，避免敌人一开始就贴近玩家或互相重叠。

为了减少“开局必输”的情况，程序还设置了 `starterEnemyCount`，保证开局有一定数量的小敌人比玩家初始体型更小，使玩家可以先通过吞噬小方块成长。

3.3 难度参数化设计

不同难度通过 `DifficultySettings` 统一管理，包括敌人数量、敌人基础大小、敌人速度、玩家速度、得分倍率、初始小敌人数量和最大道具数量。这样只需要修改难度参数，就可以调整整体游戏体验。

难度	设计特点
Easy	敌人数量较少，速度较慢，适合熟悉规则
Medium	综合难度适中，是默认难度
Tough	敌人更多，速度和分数倍率提高
Hell NO	敌人数量最多，节奏最快，容错率最低

3.4 特殊敌人行为

游戏不是只依靠普通方块增加难度，而是为敌人设计了多种差异化行为：

敌人类型	实现特点
普通敌人	按当前方向直线移动，撞到边界或敌人后改变方向
追踪敌人	根据玩家位置选择更接近玩家的移动方向
闪烁敌人	根据游戏帧数周期性改变大小
分裂敌人	被吞噬后尝试在周围生成两个更小的普通敌人
恐惧敌人	看起来可以吞噬，但合并后玩家体型会额外减少 20%
神速敌人	不受减速场影响，在减速场生效时反而追踪玩家

其中，追踪敌人会比较四个候选方向，选择既不撞墙、不撞其他敌人，又能缩短与玩家距离的方向；分裂敌人在生成子方块时也会检测玩家和已有敌人的位置，避免新生成方块立即重叠。

3.5 碰撞检测与吞噬规则

游戏中玩家、敌人和道具的碰撞主要通过矩形区域判断。`GameSquare` 类提供 `getRect()` 函数，将方块转换为 `QRect`，然后用 `intersects()` 判断是否相交。

当玩家与敌人相交时，程序根据双方边长判断结果：

条件	结果
敌人边长小于或等于玩家边长	玩家吞噬敌人，体型增大并获得得分
敌人边长大于玩家边长，且屏障未生效	游戏失败
敌人边长大于玩家边长，但屏障生效	抵消本次危险碰撞，屏障消失

玩家吞噬敌人后的新边长不是简单相加，而是根据面积合并思想计算：先将玩家和敌人的面积相加，再开平方得到新的边长。这样可以避免玩家体型增长过快，使游戏数值更加平衡。

3.6 道具系统与限时效果

玩家吞噬敌人后，道具不会必定掉落，而是按照概率生成，并受到当前难度最大道具数量限制。道具球生成后会在场上移动，碰到边界会反弹。玩家碰到道具球后，程序随机激活一种效果，并通过帧数计时控制持续时间。

道具设计不仅提供正向帮助，也加入了风险与策略。例如缩小炸弹会暂时缩小更大的敌人，给玩家创造进攻机会；但效果结束后敌人体型会放大，因此玩家需要在有限时间内决定是否趁机吞噬。

3.7 界面绘制与视觉反馈

游戏使用 `QPainter` 绘制深色背景、网格、玩家、敌人、道具和状态栏。敌人是否可吞噬通过边框颜色提示：绿色表示安全，红色表示危险。不同敌人类型还带有不同颜色和符号标记，例如追踪敌人的菱形标记、分裂敌人的十字标记、恐惧敌人的 X 标记等。

状态栏实时显示难度、玩家大小、剩余敌人数、得分、冲刺冷却和道具状态，使玩家能够根据当前局势调整策略。



四、测试与调试

在项目开发过程中，我们对游戏的核心功能和边界情况进行了多轮测试，主要包括以下内容：

测试内容	测试目的	处理结果
敌人随机生成	检查敌人是否会与玩家或其他敌人重叠	增加安全距离和重复随机机制
开局可玩性	检查是否存在开局周围全是大方块导致无法游玩的情况	设置初始小敌人数量，保证玩家有成长空间
玩家边界移动	检查玩家是否会移动出窗口	在每帧更新中限制玩家坐标范围
敌人边界移动	检查敌人是否会越界	碰到边界后改变移动方向
吞噬与失败判定	检查玩家碰到不同大小敌人时结果是否正确	根据方块边长区分吞噬、失败和屏障抵挡
道具持续时间	检查限时道具是否能按时生效和结束	使用帧数变量控制持续时间
缩小炸弹恢复	检查敌人缩小后能否恢复并放大	保存原始大小，倒计时结束后恢复更新
暂停与重开	检查暂停、继续、重新开始是否影响游戏状态	暂停时停止计时器，重开时重置状态
胜利条件	检查敌人全部被消灭后是否弹出胜利窗口	在每帧检测敌人列表是否为空

调试过程中遇到的主要问题包括随机生成敌人重叠、敌人出生点距离玩家过近、道具效果结束后状态没有及时恢复、删除敌人后敌人类型列表与敌人列表不同步等。针对这些问题，我们分别加入了位置检测、玩家安全区域、道具倒计时和统一删除函数，使游戏逻辑更加稳定。

五、小组成员分工

成员	分工
徐至晟	初步设计、初版游戏草稿、调试、大作业报告
孙广侑	功能升级、调试、游戏定稿、程序运行录屏

5.1 徐至晟 - 初版设计与文档整理

负责内容：

- 参与游戏玩法和整体规则的初步设计。
- 完成初版游戏草稿，搭建基础窗口、玩家移动和敌人方块等核心框架。
- 参与程序调试，检查基础移动、碰撞、胜负判定等功能。
- 整理大作业报告，说明项目功能、模块结构和开发反思。

5.2 孙广侑 - 功能升级与最终完善

负责内容：

- 在初版基础上进行功能升级，加入多类型敌人、道具系统、难度设置和界面优化。
- 参与程序调试，修复敌人重叠、状态恢复、列表同步等问题。
- 完成程序运行录屏。

六、项目总结与反思

本次大作业让我们较完整地体验了游戏开发流程。从最初的构思设计，到完成具备基本功能的初稿，再到后续功能升级、界面优化和游戏平衡性调整，整个过程都需要不断测试、反馈和改进。

在实现过程中，我们改进了方块随机生成算法，修复了随机生成方块可能重叠的问题；同时让敌人的生成位置与玩家保持一定距离，并保证开局时存在若干个比玩家初始体型更小的方块，从而减少“开局必输”的情况，提升了游戏的公平性和可玩性。

项目开发过程中，我们综合运用 C++ 面向对象编程知识，并学习了 Qt 的窗口绘制、事件处理、计时器、弹窗和样式表等内容。完成这个规模较大的项目不仅锻炼了我们的代码实现能力，也提升了我们将复杂任务拆分为多个模块、逐步实现并调试的能力。

团队协作也是本次项目的重要收获。成员之间通过持续沟通分配任务、反馈问题、提出新想法，使项目能够不断完善。编程和调试过程也锻炼了我们的耐心、细心和严谨性，这些经验对今后的学习和开发实践都具有价值。

附录：项目构建与运行

运行环境

项目	要求
开发语言	C++
图形框架	Qt 6
构建方式	Qt Creator 或 qmake
运行平台	Windows

构建步骤

1. 使用 Qt Creator 打开 `SquareGame/SquareGame.pro`。
2. 选择合适的 Qt Kit，例如 Desktop Qt MinGW 64-bit。
3. 点击构建并运行项目。
4. 程序启动后进入主菜单，可选择难度、查看规则或开始游戏。

核心文件结构

SquareGame/	
├─ main.cpp	# Qt 程序入口
├─ square.h	# Square 主窗口类声明
├─ square.cpp	# 游戏主体逻辑、绘制、事件和弹窗
├─ GameSquare.h	# 方块对象类声明
├─ GameSquare.cpp	# 方块对象类实现
├─ square.ui	# Qt Designer 生成的界面文件
└─ SquareGame.pro	# qmake 项目配置文件

操作流程

启动程序

- 主菜单
- 选择难度
- 查看规则或直接开始
- 控制玩家方块吞噬敌人
- 触发胜利或失败弹窗
- 选择重新开始、返回菜单或退出